

Session 8

Integration:

The Smart Garden



Teach the board to look after a plant.

Part	Topic	Time
01	The garden's senses: three sensors at once	6 min
02	From a raw number to "% wet"	7 min
03	Practice: automatic watering with a gap	10 min
04	AI integration: design the logic, choose the thresholds	7 min

Builds on calibrating the soil probe in Session 1, states from M2 Session 4, and if from Lesson 2.

Six words you will hear today.

Word	Plain meaning
Calibrate	Measure your own sensor at two known points (bone dry, in water) so its numbers mean something.
Threshold	A number where the program changes what it does — below 30 % wet, start watering.
Hysteresis	Two thresholds, one to switch on and a higher one to switch off, so the output doesn't flicker.
bool	A variable that is only ever true or false.
Fail-safe	When something goes wrong, fall back to the harmless choice — here, not watering.
Stand-in	A <code>Serial.println</code> in the spot where an actuator call will go.

A Smart Garden is a gardener's morning round.

Feel the soil — dry? Water it. Look at the sky — dark? Switch on the lamp. Too hot? Open a window.

Each sensor answers one question; your program asks them all, every round.

Three questions the garden asks.

ONE LINE OF CODE

```
float moisture = incipe.getSoilMoisture();
float temperature = incipe.getTemperature();
float light = incipe.getLightIntensity();
```

Call	Value	Not detected	Read it every
incipe.getSoilMoisture()	A float, a raw reading — no fixed unit	-1	500 ms
incipe.getTemperature()	A float in degrees Celsius (°C)	-1	2000 ms
incipe.getLightIntensity()	A float from 0 (dark) to 1023 (bright)	-1	20–500 ms

Don't assume "high = wet": calibrate your own soil probe. [Wiki › Soil moisture, Temperature & humidity, Light.](#)

Stand-ins mark where each actuator call will go.

Actuator	Job in the garden	Today's stand-in
Water pump	Water the soil	PUMP ON / PUMP OFF
Servo	Open the greenhouse window	OPEN WINDOW / CLOSE WINDOW
LED strip	Grow light	GROW LIGHT ON / OFF

Their Wiki pages name the functions but don't yet say what values they take. The logic is the hard part — and it's all here.

Two calibration readings turn any number into “% wet”.

Like a progress bar: at 30 MB of a 60 MB download you are half-way — $30 \div 60$.

Here: how far from dry $\div$ the whole way from dry to wet. Today’s code uses 100 (dry) and 40 (wet) as stand-ins — replace them with your Session 1 numbers.

Turn a raw reading into % wet.

```
1  const float DRY_VALUE = 100;    // probe in air: change to yours
2  const float WET_VALUE = 40;     // probe in water: change to yours
3
4  float wetPercent(float raw) {
5      return (raw - DRY_VALUE) / (WET_VALUE - DRY_VALUE) * 100;
6  }
```

LINE 1 • STEP 1 OF 6

Your reading with the probe in the air: bone dry.

VARIABLES

DRY_VALUE = 100

Turn a raw reading into % wet.

```
1  const float DRY_VALUE = 100;    // probe in air: change to yours
2  const float WET_VALUE = 40;     // probe in water: change to yours
3
4  float wetPercent(float raw) {
5      return (raw - DRY_VALUE) / (WET_VALUE - DRY_VALUE) * 100;
6  }
```

LINE 2 • STEP 2 OF 6

Your reading with the probe in water: fully wet.

VARIABLES

DRY_VALUE = 100

WET_VALUE = 40

Turn a raw reading into % wet.

```
1  const float DRY_VALUE = 100;    // probe in air: change to yours
2  const float WET_VALUE = 40;     // probe in water: change to yours
3
4  float wetPercent(float raw) {
5      return (raw - DRY_VALUE) / (WET_VALUE - DRY_VALUE) * 100;
6  }
```

LINE 4 • STEP 3 OF 6

Call `wetPercent(70)`: a reading between the two.

VARIABLES

`DRY_VALUE = 100`

`WET_VALUE = 40`

`raw = 70`

Turn a raw reading into % wet.

```
1  const float DRY_VALUE = 100;    // probe in air: change to yours
2  const float WET_VALUE = 40;     // probe in water: change to yours
3
4  float wetPercent(float raw) {
5      return (raw - DRY_VALUE) / (WET_VALUE - DRY_VALUE) * 100;
6  }
```

LINE 5 • STEP 4 OF 6

How far from dry? $70 - 100 = -30$.

VARIABLES

DRY_VALUE = 100

WET_VALUE = 40

raw = 70

Turn a raw reading into % wet.

```
1  const float DRY_VALUE = 100;    // probe in air: change to yours
2  const float WET_VALUE = 40;     // probe in water: change to yours
3
4  float wetPercent(float raw) {
5      return (raw - DRY_VALUE) / (WET_VALUE - DRY_VALUE) * 100;
6  }
```

LINE 5 • STEP 5 OF 6

The whole way from dry to wet: $40 - 100$
 $= -60$.

VARIABLES

DRY_VALUE = 100

WET_VALUE = 40

raw = 70

Turn a raw reading into % wet.

```
1  const float DRY_VALUE = 100;    // probe in air: change to yours
2  const float WET_VALUE = 40;     // probe in water: change to yours
3
4  float wetPercent(float raw) {
5      return (raw - DRY_VALUE) / (WET_VALUE - DRY_VALUE) * 100;
6  }
```

LINE 5 • STEP 6 OF 6

$-30 \div -60 = 0.5$, then $\times 100 = 50$. Half-way to wet.

VARIABLES

DRY_VALUE = 100

WET_VALUE = 40

raw = 70

result = 50

One threshold makes the pump flicker.

“Water below 30 %”: at 30.1 % it stops, a minute later at 29.9 % it starts again — on, off, on, off.

Like running a bath: tap on when the water is low, off only when it is high enough. Two levels — that gap is hysteresis.

Two states, two thresholds.

State	Switch when	Next state	Stand-in
Not watering	% wet below 30	Watering	PUMP ON
Watering	% wet above 60	Not watering	PUMP OFF

A two-state machine, like the traffic light in M2 Session 4. A bool remembers which state you're in.

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 1 · STEP 1 OF 14

Start watering below 30 % wet.

VARIABLES

START = 30

SERIAL MONITOR

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 2 · STEP 2 OF 14

Stop only above 60 % wet.

VARIABLES

START = 30

STOP = 60

SERIAL MONITOR

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 3 · STEP 3 OF 14

The pump starts off.

VARIABLES

watering = `false`

SERIAL MONITOR

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6    float raw = incipe.getSoilMoisture();
7    float wet = 100;          // no reading: act as if wet
8    if (raw != -1) wet = wetPercent(raw);
9    if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10   if (watering && wet > STOP)   { watering = false; Serial.println("PUMP OFF"); }
11   delay(2000);
12 }
```

LINE 6 · STEP 4 OF 14

Round 1: the reading is 70.

VARIABLES

```
watering = false
raw = 70
wet = ?
```

SERIAL MONITOR

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 8 · STEP 5 OF 14

Not -1, so convert: 50 % wet.

VARIABLES

```
watering = false
raw = 70
wet = 50
```

SERIAL MONITOR

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 9 · STEP 6 OF 14

!watering means “not watering” — true. Is 50 below 30? No.

VARIABLES

```
watering = false
raw = 70
wet = 50
```

SERIAL MONITOR

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 10 · STEP 7 OF 14

Not watering, so this line does nothing either. Wait 2 s.

VARIABLES

```
watering = false
raw = 70
wet = 50
```

SERIAL MONITOR

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 8 · STEP 8 OF 14

Round 2: the reading is 85 — 25 % wet. The soil is drying.

VARIABLES

watering = false
raw = 85
wet = 25

SERIAL MONITOR

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 9 · STEP 9 OF 14

Not watering, and 25 is below 30: switch on.

VARIABLES

watering = **true**
raw = 85
wet = 25

SERIAL MONITOR

PUMP ON

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 10 · STEP 10 OF 14

Is 25 above 60? No — keep going.

VARIABLES

```
watering = true
raw = 85
wet = 25
```

SERIAL MONITOR

```
PUMP ON
```

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 8 · STEP 11 OF 14

Round 3: the reading is 73 — 45 % wet.

VARIABLES

watering = true
raw = 73
wet = 45

SERIAL MONITOR

PUMP ON

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 10 · STEP 12 OF 14

45 is above 30 now, but not above 60: the pump keeps running. That's the gap at work.

VARIABLES

```
watering = true
raw = 73
wet = 45
```

SERIAL MONITOR

```
PUMP ON
```

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 8 · STEP 13 OF 14

Round 4: the reading is 61 — 65 % wet.

VARIABLES

watering = true
raw = 61
wet = 65

SERIAL MONITOR

PUMP ON

Automatic watering, round by round.

```
1  const float START = 30;    // % wet: start watering below this
2  const float STOP = 60;     // % wet: stop watering above this
3  bool watering = false;
4
5  void loop() {
6      float raw = incipe.getSoilMoisture();
7      float wet = 100;        // no reading: act as if wet
8      if (raw != -1) wet = wetPercent(raw);
9      if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }
10     if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }
11     delay(2000);
12 }
```

LINE 10 · STEP 14 OF 14

Watering, and 65 is above 60: switch off.

VARIABLES

watering = **false**
raw = 61
wet = 65

SERIAL MONITOR

PUMP ON
PUMP OFF

Make it your garden.

Step	Do this
1	Put in your own calibration values. Pick START and STOP for a real plant.
2	Grow light: print GROW LIGHT ON when the light falls below a threshold you choose (0 dark, 1023 bright), and OFF above a higher one. Use a second bool. Skip -1.
3	Window: print OPEN WINDOW above 30 °C and CLOSE WINDOW below 27 °C.
4	Pull the soil probe out while it runs. What does the Serial Monitor show, and why is that safe?

Ask the AI to design the logic and suggest thresholds.

PASTE YOUR SKETCH AND CALIBRATION INTO THE WORKSPACE AI CHAT WITH

I'm building a Smart Garden on an ESP32 board. My soil probe reads [DRY] in air and [WET] in water; I convert it to "% wet" with the function below. Suggest START and STOP thresholds for watering a [type of plant], and a light threshold for a grow lamp on a 0 (dark) to 1023 (bright) scale. Explain each number in one sentence for a beginner. Keep my hysteresis and my -1 fail-safe. Do not change any incipe.* call.

Check what comes back like a reviewer.

Ask yourself	Why
Are its thresholds in % wet, not raw numbers?	The AI can't know your probe's raw range — only your calibration does.
Is STOP still higher than START?	Without the gap, the pump flickers again.
Is the -1 check still there?	A missing probe must never start the pump.
Did it invent pump, servo or LED code, or change an incipe.* call?	Never accept that — compare with the Wiki. Keep your stand-ins.
Does the plant advice make sense?	Ask it why — then check one fact yourself.

Checkpoint 1

With DRY 100 and WET 40, what does wetPercent(100) return?

READ THE CODE

```
(100 - 100) / (40 - 100) * 100
```

ANSWER

0

WHY

The reading equals “dry”.

And wetPercent(40)?

READ THE CODE

```
(40 - 100) / (40 - 100) * 100
```

ANSWER

100

WHY

The reading equals “wet”.

Checkpoint 3

watering is false and the soil is 45 % wet. Does the pump start?

READ THE CODE

```
if (!watering && wet < START)
    ...
// START = 30
```

ANSWER

No

WHY

45 is not below START (30), so line 9 does nothing.

Checkpoint 4

watering is true and the soil is 45 % wet. What happens?

READ THE CODE

```
if (watering && wet > STOP)
    ...
// STOP = 60
```

ANSWER

Keeps running

WHY

Same reading, different state, different result — that's hysteresis.

Checkpoint 5

If line 8 converted -1 anyway, what would wetPercent(-1) print?

READ THE CODE

```
(-1 - 100) / (40 - 100) * 100
```

ANSWER

168.33

WHY

A nonsense “very wet”. A probe going the other way would give a nonsense “very dry” and start the pump.

Three things to take away.

1

Calibrate, then convert: two readings turn any raw number into “% wet”, whichever way your probe goes.

2

Two thresholds — start low, stop high — and a bool for the state, so the pump switches once instead of flickering.

3

Fail safe: –1 never switches anything on. The AI can suggest thresholds; only your calibration knows your sensor.

Next — Session 9: Building the Game Console. Joystick, button and LED strip together.